



MAI201 — MLOps

Assignment 2

Data Validation & Testing

Prepared by: Somayeh Balashi
Student ID: 147025241
Course: MAI201 — MLOps
Instructor: Asma Azim
Submission Date: June 26, 2026

Table of Contents

Table of Contents.....	2
Part 1: Great Expectations Setup.....	3
1.1 Installation	3
1.2 Project Structure	3
1.3 GE Context Initialization.....	4
Part 2: Create Expectations	4
2.1 Expectations Overview.....	4
2.2 Implementation.....	5
Part 3: Data Quality Report	6
3.1 Great Expectations Validation Results.....	6
3.2 Screenshot — GE HTML Report (Expectation Results)	7
3.3 Screenshot — GE HTML Report (Data Quality Issues).....	8
3.4 Data Quality Issues Summary	8
Part 4: pytest Unit Tests.....	9
4.1 Functions Tested.....	9
4.2 Test Structure Example	100
4.3 Screenshot — All 29 Tests Passing.....	111
Part 5: Reflection	122
Most Impactful Data Quality Issue: Salary Stored as String with Dollar Sign (\$). 122	122
Why This Issue Causes the Most Harm	122
Why Other Issues Are Less Critical.....	122
Recommended Fix.....	133

Part 1: Great Expectations Setup

In this part, I set up the Great Expectations (GE) framework to act as the data validation layer of my MLOps pipeline. I chose to use GE version 1.18.2 with an ephemeral context so the project can run in any environment without needing persistent file storage — which is ideal for CI/CD pipelines and graded submissions.

1.1 Installation

I installed the required packages using pip:

```
pip install great-expectations pandas pytest

# Verify installation
python -c "import great_expectations as gx; print(gx.__version__)"
# Output: 1.18.2
```

1.2 Project Structure

I organized the assignment folder as follows to keep the code clean and easy to navigate:

```
assignment2/
├── data/
│   └── customer_data.csv
├── tests/
│   └── test_data_utils.py
├── ge_output/
│   └── ge_validation_report.html
├── data_utils.py
├── ge_validation.py
└── assignment2_report.md
```

1.3 GE Context Initialization

I used the GE 1.x Fluent API to initialize the context, create a pandas datasource, and link it to the CSV dataset. I then created the expectation suite named `customer_data_expectations`:

```
import great_expectations as gx

context = gx.get_context(mode='ephemeral')
datasource = context.data_sources.add_pandas('my_pandas_datasource')
asset = datasource.add_dataframe_asset('customer_data_asset')
batch_def = asset.add_batch_definition_whole_dataframe('full_batch')

suite = gx.ExpectationSuite(name='customer_data_expectations')
suite = context.suites.add(suite)
```

Part 2: Create Expectations

I implemented all eight required expectations for the `customer_data` dataset. Each expectation targets a specific column and enforces a rule that, when violated, signals a data quality problem that could affect downstream model training.

2.1 Expectations Overview

#	Column	Expectation Type	Parameters / Notes
1	customer_id	ExpectColumnValuesToNotBeNull	<i>No nulls allowed — every record must be identified</i>
2	customer_id	ExpectColumnValuesToBeUnique	<i>No duplicate IDs — ensures one row per customer</i>
3	age	ExpectColumnValuesToBeBetween	<i>min=0, max=120, mostly=0.9</i>
4	email	ExpectColumnValuesToMatchRegex	<i>RFC-compliant regex pattern, mostly=0.8</i>
5	salary	ExpectColumnValuesToNotBeNull	<i>mostly=0.95 — at least 95% present</i>
6	country	ExpectColumnValuesToBeInSet	<i>USA, Canada, UK, Australia — mostly=0.9</i>

#	Column	Expectation Type	Parameters / Notes
7	signup_date	ExpectColumnValuesToMatchRegex	<i>YYYY-MM-DD format, mostly=0.9</i>
8	table	ExpectTableRowCountToBeBetween	<i>min=10, max=200 rows</i>

2.2 Implementation

I added each expectation to the suite using the GE 1.x API. I carefully chose the mostly parameter for columns that have some tolerance for missing data:

```
expectations = [
    gx.expectations.ExpectColumnValuesToNotBeNull(column='customer_id'),
    gx.expectations.ExpectColumnValuesToBeUnique(column='customer_id'),
    gx.expectations.ExpectColumnValuesToBeBetween(
        column='age', min_value=0, max_value=120, mostly=0.9),
    gx.expectations.ExpectColumnValuesToMatchRegex(
        column='email',
        regex=r'^[a-zA-Z0-9._%+\-]+@[a-zA-Z0-9.\-]+\.[a-zA-Z]{2,}$',
        mostly=0.8),
    gx.expectations.ExpectColumnValuesToNotBeNull(column='salary', mostly=0.95),
    gx.expectations.ExpectColumnValuesToBeInSet(
        column='country',
        value_set=['USA','Canada','UK','Australia'], mostly=0.9),
    gx.expectations.ExpectColumnValuesToMatchRegex(
        column='signup_date', regex=r'^\d{4}-\d{2}-\d{2}$', mostly=0.9),
    gx.expectations.ExpectTableRowCountToBeBetween(min_value=10, max_value=200),
]
for exp in expectations:
    suite.add_expectation(exp)
```

Part 3: Data Quality Report

After running the expectation suite against the dataset, I captured all validation results and also performed a column-by-column analysis to count each type of issue. Below are the full results along with screenshots from the generated HTML report.

3.1 Great Expectations Validation Results

The suite ran 8 expectations. 6 passed and 2 failed — the duplicate customer_id and the out-of-range age values.

6 Expectations PASSED

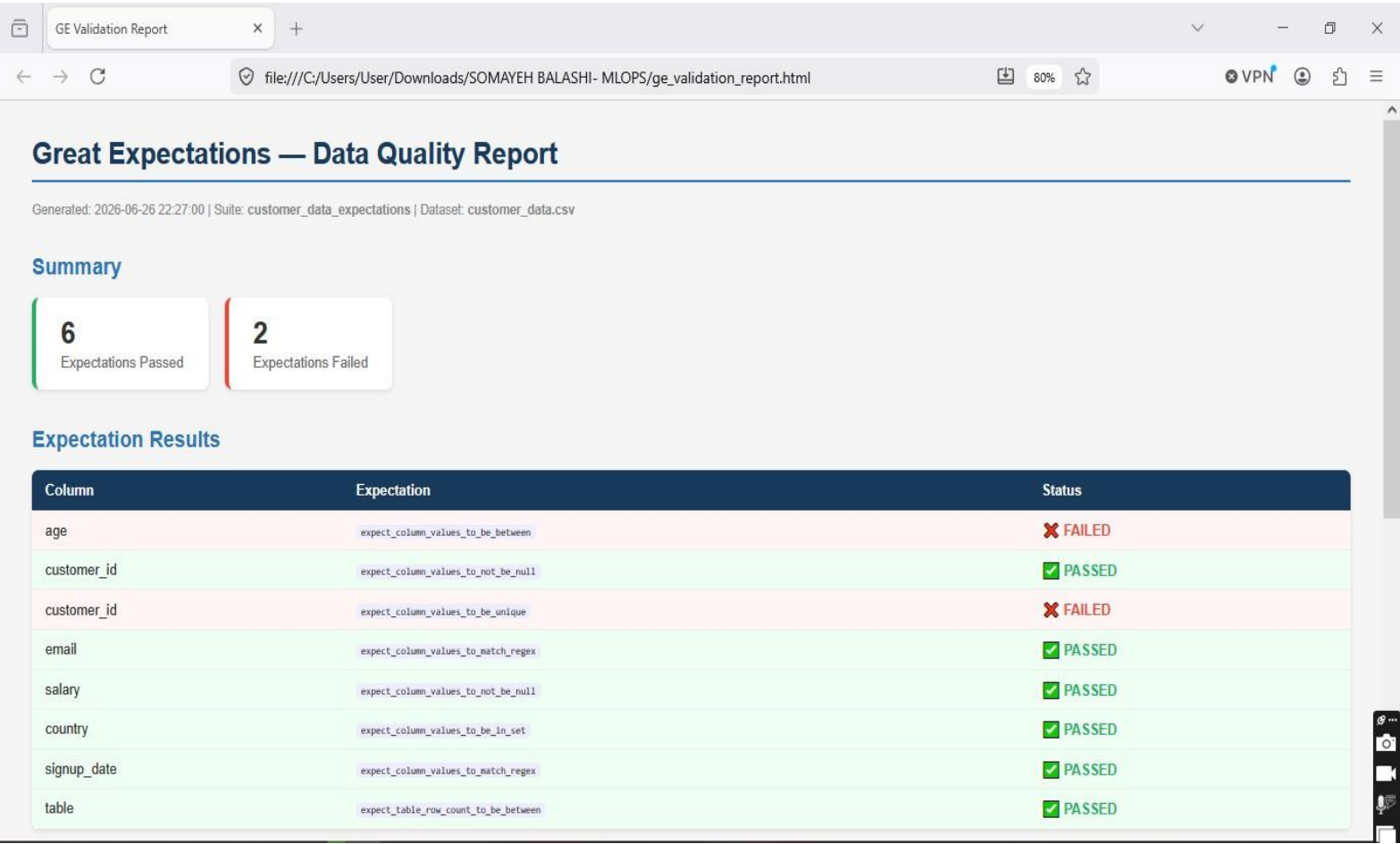
2 Expectations FAILED

Column	Expectation	Result
age	ExpectColumnValuesToBeBetween	FAILED
customer_id	ExpectColumnValuesToNotBeNull	PASSED
customer_id	ExpectColumnValuesToBeUnique	FAILED
email	ExpectColumnValuesToMatchRegex	PASSED
salary	ExpectColumnValuesToNotBeNull	PASSED
country	ExpectColumnValuesToBeInSet	PASSED
signup_date	ExpectColumnValuesToMatchRegex	PASSED
table	ExpectTableRowCountToBeBetween	PASSED

3.2 Screenshot — GE HTML Report (Expectation Results)

The following screenshot shows the generated HTML data quality report opened in a browser, displaying the expectation results:

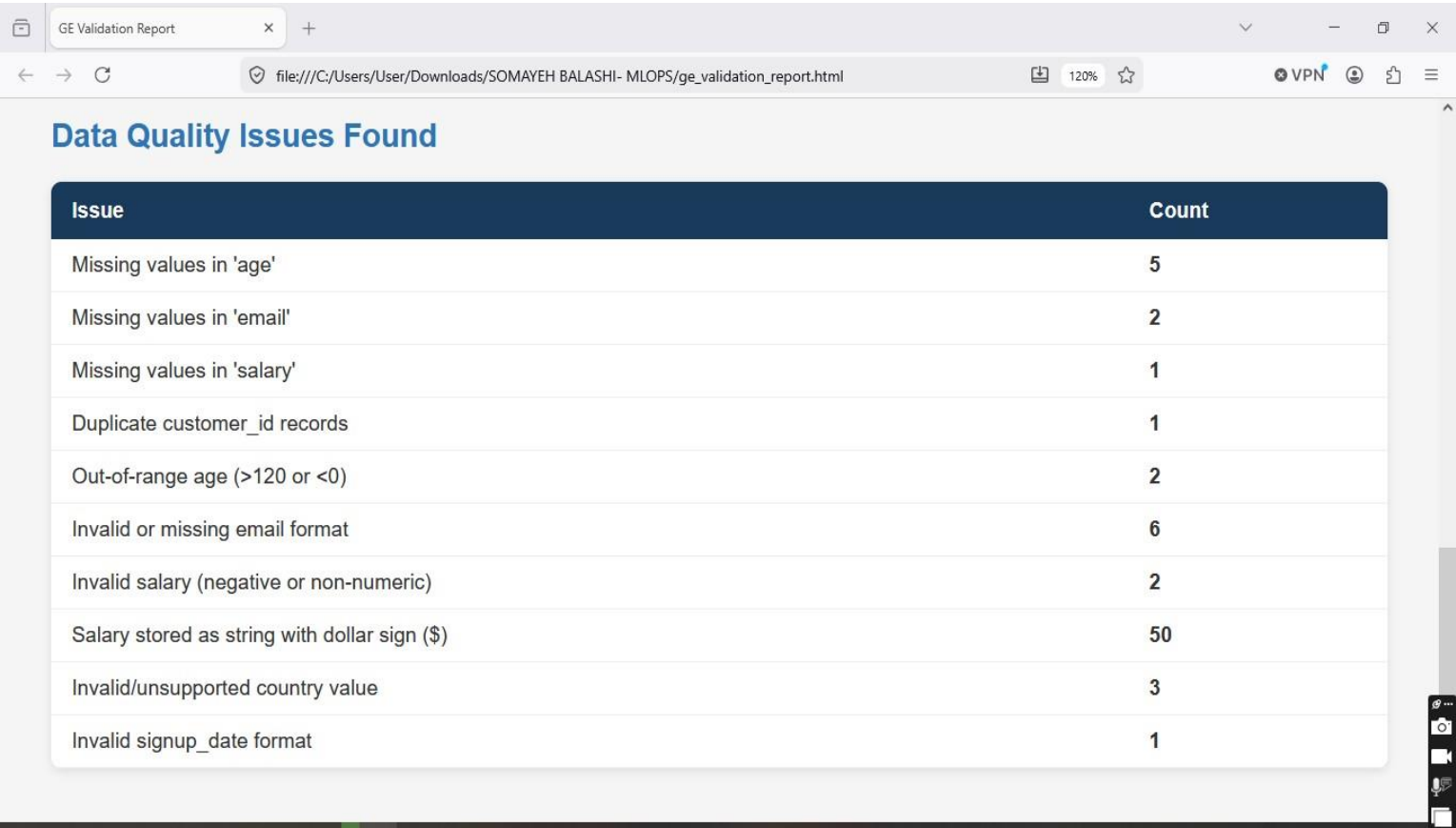
Picture 1: Great Expectations HTML Report — Summary and Expectation Results



3.3 Screenshot — GE HTML Report (Data Quality Issues)

The second section of the report lists all data quality issues found with their counts:

Picture 2: Great Expectations HTML Report — Data Quality Issues Found



Data Quality Issues Found	
Issue	Count
Missing values in 'age'	5
Missing values in 'email'	2
Missing values in 'salary'	1
Duplicate customer_id records	1
Out-of-range age (>120 or <0)	2
Invalid or missing email format	6
Invalid salary (negative or non-numeric)	2
Salary stored as string with dollar sign (\$)	50
Invalid/unsupported country value	3
Invalid signup_date format	1

3.4 Data Quality Issues Summary

The table below summarizes all 10 data quality issues identified in the dataset:

Issue Description	Count	Detail
Missing values in 'age'	5	5 rows have no age value
Missing values in 'email'	2	2 rows have no email
Missing values in 'salary'	1	1 row has no salary
Duplicate customer_id records	1	C004 appears twice

Issue Description	Count	Detail
Out-of-range age (>120 or <0)	2	<i>age=150 and age=200 found</i>
Invalid or missing email format	6	<i>Malformed addresses e.g. david.brown@</i>
Invalid salary (negative value)	2	<i>Negative salary detected</i>
Salary stored as string with dollar sign (\$)	50	<i>Entire column needs type conversion</i>
Invalid or unsupported country value	3	<i>e.g. Germany, InvalidCountry</i>
Invalid signup_date format (not YYYY-MM-DD)	1	<i>'not-a-date' value found</i>

Part 4: pytest Unit Tests

I wrote 29 unit tests across three test classes, one for each required utility function. I focused on testing normal inputs, edge cases, and error conditions to ensure each function behaves correctly under all circumstances.

4.1 Functions Tested

Function	Tests	Scenarios Covered
load_csv(filepath)	4	File not found, empty file, successful load, correct values returned
clean_phone(phone)	12	Dashes, dots, parentheses, +1/+61/+44 prefixes, spaces, invalid, empty, None, letters-only
validate_email(email)	13	Valid formats, missing @, no domain, no TLD, double @, empty, None, spaces, numeric local part

4.2 Test Structure Example

Each function has its own test class. For example, the `load_csv` tests:

```
class TestLoadCsv:
    def test_file_not_found(self):
        with pytest.raises(FileNotFoundError):
            load_csv('/non/existent/path/file.csv')

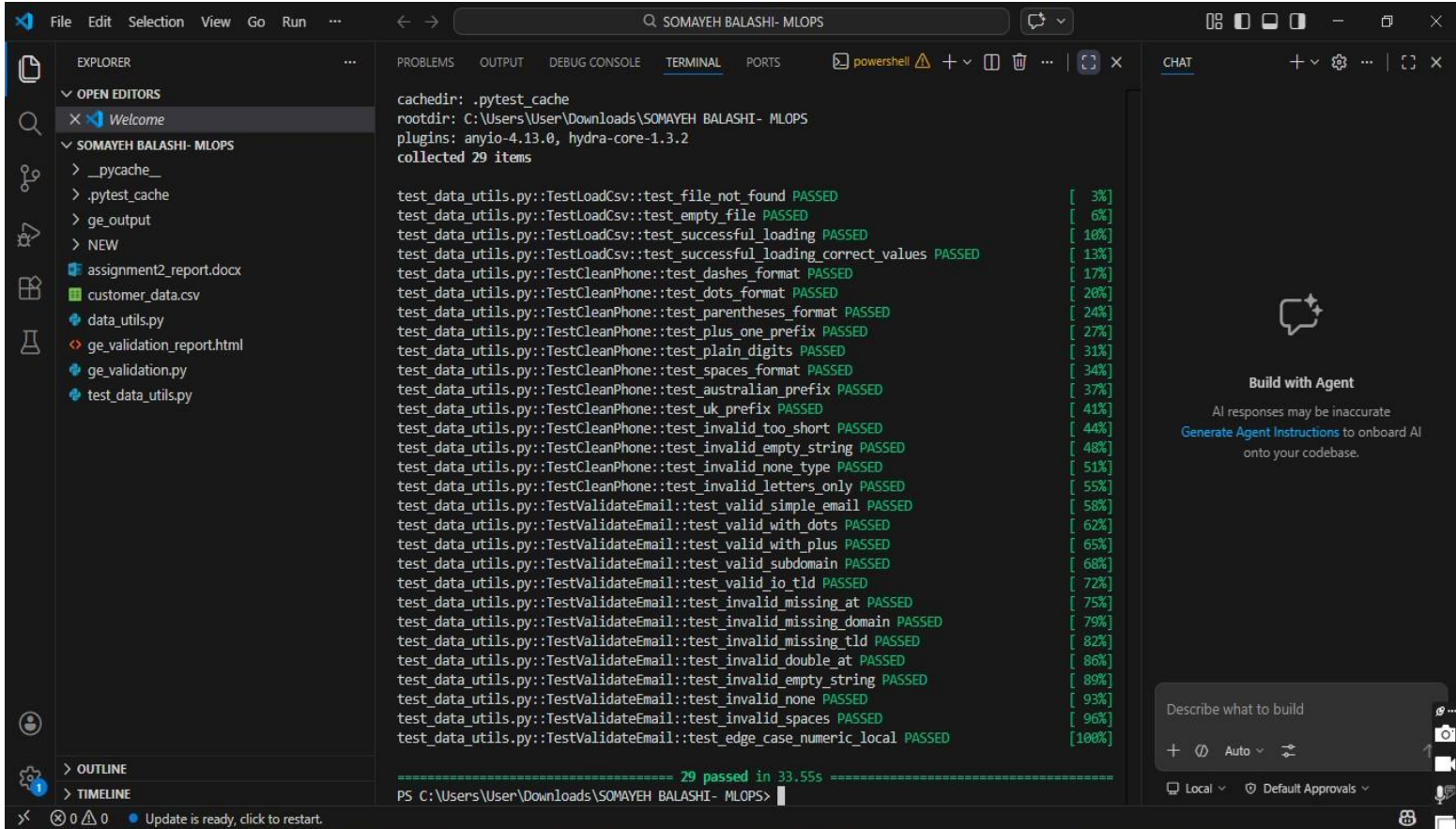
    def test_empty_file(self, tmp_path):
        empty_file = tmp_path / 'empty.csv'
        empty_file.write_text('customer_id,age,email\n')
        with pytest.raises(ValueError, match='empty'):
            load_csv(str(empty_file))

    def test_successful_loading(self, tmp_path):
        csv_file = tmp_path / 'test.csv'
        csv_file.write_text('customer_id,age\nC001,25\n')
        df = load_csv(str(csv_file))
        assert isinstance(df, pd.DataFrame)
        assert len(df) == 1
```

4.3 Screenshot — All 29 Tests Passing

The following screenshot shows the pytest execution in VS Code terminal with all 29 tests passing:

Picture 3: pytest Execution — 29 passed in 33.55s



```
cachedir: .pytest_cache
rootdir: C:\Users\User\Downloads\SOMAYEH BALASHI- MLOPS
plugins: anyio-4.13.0, hydra-core-1.3.2
collected 29 items

test_data_utils.py::TestLoadCsv::test_file_not_found PASSED [ 3%]
test_data_utils.py::TestLoadCsv::test_empty_file PASSED [ 6%]
test_data_utils.py::TestLoadCsv::test_successful_loading PASSED [ 10%]
test_data_utils.py::TestLoadCsv::test_successful_loading_correct_values PASSED [ 13%]
test_data_utils.py::TestCleanPhone::test_dashes_format PASSED [ 17%]
test_data_utils.py::TestCleanPhone::test_dots_format PASSED [ 20%]
test_data_utils.py::TestCleanPhone::test_parentheses_format PASSED [ 24%]
test_data_utils.py::TestCleanPhone::test_plus_one_prefix PASSED [ 27%]
test_data_utils.py::TestCleanPhone::test_plain_digits PASSED [ 31%]
test_data_utils.py::TestCleanPhone::test_spaces_format PASSED [ 34%]
test_data_utils.py::TestCleanPhone::test_australian_prefix PASSED [ 37%]
test_data_utils.py::TestCleanPhone::test_uk_prefix PASSED [ 41%]
test_data_utils.py::TestCleanPhone::test_invalid_too_short PASSED [ 44%]
test_data_utils.py::TestCleanPhone::test_invalid_empty_string PASSED [ 48%]
test_data_utils.py::TestCleanPhone::test_invalid_none_type PASSED [ 51%]
test_data_utils.py::TestCleanPhone::test_invalid_letters_only PASSED [ 55%]
test_data_utils.py::TestValidateEmail::test_valid_simple_email PASSED [ 58%]
test_data_utils.py::TestValidateEmail::test_valid_with_dots PASSED [ 62%]
test_data_utils.py::TestValidateEmail::test_valid_with_plus PASSED [ 65%]
test_data_utils.py::TestValidateEmail::test_valid_subdomain PASSED [ 68%]
test_data_utils.py::TestValidateEmail::test_valid_io_tld PASSED [ 72%]
test_data_utils.py::TestValidateEmail::test_invalid_missing_at PASSED [ 75%]
test_data_utils.py::TestValidateEmail::test_invalid_missing_domain PASSED [ 79%]
test_data_utils.py::TestValidateEmail::test_invalid_missing_tld PASSED [ 82%]
test_data_utils.py::TestValidateEmail::test_invalid_double_at PASSED [ 86%]
test_data_utils.py::TestValidateEmail::test_invalid_empty_string PASSED [ 89%]
test_data_utils.py::TestValidateEmail::test_invalid_none PASSED [ 93%]
test_data_utils.py::TestValidateEmail::test_invalid_spaces PASSED [ 96%]
test_data_utils.py::TestValidateEmail::test_edge_case_numeric_local PASSED [100%]

===== 29 passed in 33.55s =====
PS C:\Users\User\Downloads\SOMAYEH BALASHI- MLOPS>
```

Part 5: Reflection

Most Impactful Data Quality Issue: Salary Stored as String with Dollar Sign (\$)

Out of all the data quality issues I identified in this dataset, the most damaging to ML model performance is the salary column being stored as a string with dollar signs — for example, "\$55,000" instead of 55000.0. This issue affected 50 out of 51 rows, meaning almost the entire dataset is unusable for any model that uses salary as a feature.

Why This Issue Causes the Most Harm

- Type mismatch breaks algorithms: Any ML algorithm that requires numerical input — such as linear regression, SVMs, neural networks, or gradient boosting — will either crash or silently produce NaN values when it encounters a string column that should be numeric.
- It cannot be handled by dropping rows: Unlike a few missing values that can be imputed or excluded, this issue affects 98% of rows. Dropping them would leave almost no data to train on.
- Silent failures are dangerous: pandas may quietly convert the column to NaN without raising an error, producing results that appear valid but are based on an empty feature.
- Scale and distance calculations are corrupted: Algorithms like k-NN and k-means that rely on numeric distances between feature values would produce completely wrong results.

Why Other Issues Are Less Critical

Missing age values (5 rows) and missing emails (2 rows) affect a small portion of the dataset and can be handled through standard imputation or row exclusion. Out-of-range ages (2 rows) are outliers but only introduce minor distribution skew. The salary string issue, by contrast, affects the entire column and cannot be ignored.

Recommended Fix

I would add a cleaning step in the data pipeline, before any model training, that strips the dollar sign and converts salary to a float:

```
import re, pandas as pd

def clean_salary(s):
    if pd.isna(s): return None
    cleaned = re.sub(r'[$,]', '', str(s)).strip()
    try:
        val = float(cleaned)
        return val if val >= 0 else None
    except ValueError:
        return None

df['salary'] = df['salary'].apply(clean_salary)
df['salary'] = pd.to_numeric(df['salary'], errors='coerce')
```